

HR Schema Insights

Simple Queries for Beginners

Tank54 Group | Oracle HR Schema

This guide shows what useful information you can get from the HR database using simple queries. Part 1-5 use 1 or 2 tables. Part 6 uses 3 or more tables. We use INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, and WHERE clause. Each query answers one simple business question.

English level: A1 (easy words, short sentences)

The Tables We Use

In this guide we use only 4 tables from the HR database. Here is what each table has:

Table	What It Has	Key Columns
EMPLOYEES	All workers in the company	first_name, last_name, salary, department_id, manager_id, hire_date, job_id, commission_pct
DEPARTMENTS	Company departments (teams)	department_id, department_name, manager_id, location_id
JOBS	Job titles (roles) and pay range	job_id, job_title, min_salary, max_salary
LOCATIONS	Office cities and countries	location_id, city, country_id
COUNTRIES	Country names and region	country_id, country_name, region_id
REGIONS	World regions	region_id, region_name

How Tables Connect:

- EMPLOYEES and DEPARTMENTS connect by DEPARTMENT_ID
- EMPLOYEES and JOBS connect by JOB_ID
- DEPARTMENTS and LOCATIONS connect by LOCATION_ID
- EMPLOYEES and EMPLOYEES connect by MANAGER_ID (a manager is also an employee)
- LOCATIONS and COUNTRIES connect by COUNTRY_ID
- COUNTRIES and REGIONS connect by REGION_ID

Part 1: INNER JOIN Insights

INNER JOIN shows only rows that match in both tables. Use it when you want only rows that have a match on both sides.

Insight 1: Employee Name and Department

Business Question:

What is the name of each employee and which department do they work in?

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
ORDER BY d.department_name;
```

How It Works:

We take EMPLOYEES and DEPARTMENTS. The shared column is DEPARTMENT_ID. INNER JOIN shows only employees who have a department. If an employee has no department, they do not appear. The result has two names from two tables, but in one row. This is the most common JOIN query in real work.

Result:

First Name	Last Name	Department
Diana	Lorentz	IT
Steven	King	Executive
Neena	Kochhar	Executive
Shelley	Higgins	Accounting

Remember: "e" is a short name for employees. "d" is a short name for departments. We use short names to make the query shorter and easier to write.

Insight 2: Employee Name and Job Title

Business Question:

What is the name and job title (role) of each employee?

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       j.job_title
FROM employees e
INNER JOIN jobs j
    ON e.job_id = j.job_id
ORDER BY j.job_title;
```

How It Works:

We take EMPLOYEES and JOBS. The shared column is JOB_ID. Every employee has a job_id. So every employee appears in the result. This query shows what role each person has in the company. For example: Steven is the President, Diana is a Programmer.

Result:

First Name	Last Name	Job Title
Steven	King	President
Neena	Kochhar	Administration Vice President
Diana	Lorentz	Programmer
Jennifer	Whalen	Administration Assistant

Insight 3: Department Name and City

Business Question:

Where is each department located? Show the department name and the city.

SQL Query:

```
SELECT d.department_name,
       l.city
FROM departments d
INNER JOIN locations l
      ON d.location_id = l.location_id
ORDER BY l.city;
```

How It Works:

We take DEPARTMENTS and LOCATIONS. The shared column is LOCATION_ID. Each department has a location_id. So every department appears. This shows which city each team works in. For example: IT is in South San Francisco, Executive is in Seattle.

Result:

Department	City
IT	South San Francisco
Executive	Seattle
Sales	Seattle
Marketing	Toronto

Insight 4: Employees in One Department (WHERE)

Business Question:

Who works in the IT department? Show their name and salary.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM employees e
INNER JOIN departments d
      ON e.department_id = d.department_id
WHERE d.department_name = 'IT';
```

How It Works:

First we join EMPLOYEES with DEPARTMENTS to get the department name. Then we use WHERE to keep only the rows where department_name is "IT". WHERE is a filter. It keeps only the rows you want. You can change "IT" to any other department name to see its employees.

Result:

First Name	Last Name	Salary
Alexander	Hunold	9000
Bruce	Ernst	6000
David	Austin	4800
Valli	Pataballa	4800
Diana	Lorentz	4200

Tip: You can change 'IT' to any department name to see employees in that department. For example: 'Sales', 'Executive', 'Marketing'.

Insight 5: Employees With High Salary (WHERE)

Business Question:

Which employees earn more than 10000? Show their name, salary, and job title.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       e.salary,
       j.job_title
FROM employees e
INNER JOIN jobs j
      ON e.job_id = j.job_id
WHERE e.salary > 10000
ORDER BY e.salary DESC;
```

How It Works:

We join EMPLOYEES with JOBS to see the job title. WHERE e.salary > 10000 keeps only employees whose salary is bigger than 10000. ORDER BY e.salary DESC shows the highest salary at the top. DESC means "from big to small". You can change 10000 to any number to find different salary levels.

Result:

First Name	Last Name	Salary	Job Title
Steven	King	24000	President
Neena	Kochhar	17000	Administration VP
Lex	De Haan	17000	Administration VP
Shelley	Higgins	12008	Accounting Manager

Tip: You can use other operators in WHERE:

- e.salary < 5000 (less than 5000)
- e.salary >= 10000 (10000 or more)
- e.salary != 24000 (not equal to 24000)
- e.salary BETWEEN 5000 AND 10000 (between 5000 and 10000)

Part 2: LEFT JOIN Insights

LEFT JOIN shows ALL rows from the left table (the first table). If a row has no match in the right table, it shows null. Use it when you want to keep ALL rows from the first table.

Insight 6: All Employees With Their Department

Business Question:

Show ALL employees and their department. Include employees who have no department.

SQL Query:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
LEFT JOIN departments d  
  ON e.department_id = d.department_id  
ORDER BY d.department_name;
```

How It Works:

We use LEFT JOIN because we want ALL employees. EMPLOYEES is the left table (after FROM). If an employee has no department, we still see the employee name but the department name will be null. Compare this with Insight 1 (INNER JOIN). INNER JOIN hides employees with no department. LEFT JOIN keeps them.

Result:

First Name	Last Name	Department
Steven	King	Executive
Diana	Lorentz	IT
Kimberely	Grant	null

Kimberely has no department. With INNER JOIN she is hidden. With LEFT JOIN she appears with null.

Insight 7: Employees Without a Department

Business Question:

Are there employees who have NO department? Show their name.

SQL Query:

```
SELECT e.first_name,  
       e.last_name  
FROM employees e  
LEFT JOIN departments d  
  ON e.department_id = d.department_id  
WHERE d.department_id IS NULL;
```

How It Works:

First we use LEFT JOIN to keep all employees. If an employee has no department, the department_id from DEPARTMENTS will be null. Then WHERE d.department_id IS NULL keeps only those rows where the department is null. This means: "Show me only the employees who have no match in DEPARTMENTS." This is a data quality check. Every employee should have a department. If this query returns rows, there is a data problem to fix.

Result:

First Name	Last Name
Kimberely	Grant

Key Pattern: LEFT JOIN + WHERE ... IS NULL = find rows with NO match.
This is one of the most useful patterns in SQL.

Insight 8: Employees Hired After a Date (WHERE)

Business Question:

Show employees who started work after 1 January 2005. Show name, hire date, and department.

SQL Query:

```
SELECT e.first_name,  
       e.last_name,
```

```
e.hire_date,  
d.department_name  
FROM employees e  
LEFT JOIN departments d  
    ON e.department_id = d.department_id  
WHERE e.hire_date > '01-JAN-2005'  
ORDER BY e.hire_date;
```

How It Works:

We use LEFT JOIN to keep all employees, even those without a department. WHERE e.hire_date > '01-JAN-2005' keeps only employees hired after that date. The date format in Oracle is DD-MON-YYYY. You can change the date to any date you want. This query helps HR see who is new in the company.

Result:

First Name	Last Name	Hire Date	Department
Kimberely	Grant	24-MAY-2007	null
Douglas	Grant	13-JAN-2008	Sales
Michael	Hartstein	17-FEB-2004	Marketing

Tip: You can also use these date comparisons:

- e.hire_date >= '01-JAN-2005' (on or after)
- e.hire_date < '01-JAN-2005' (before)
- e.hire_date BETWEEN '01-JAN-2005' AND '31-DEC-2006' (between two dates)

Part 3: RIGHT JOIN Insights

RIGHT JOIN shows ALL rows from the right table (the second table). If a row has no match in the left table, it shows null. Use it when you want to keep ALL rows from the second table.

Insight 9: All Departments and Their Employees

Business Question:

Show ALL departments and the employees who work there. Include departments with no employees.

SQL Query:

```
SELECT d.department_name,  
       e.first_name,  
       e.last_name  
FROM employees e  
RIGHT JOIN departments d  
      ON e.department_id = d.department_id  
ORDER BY d.department_name;
```

How It Works:

We use RIGHT JOIN because we want ALL departments. DEPARTMENTS is the right table (after JOIN). If a department has no employees, we still see the department name but the employee name will be null. This shows us which departments are empty (have no people). This is useful because an empty department costs money but has no workers.

Result:

Department	First Name	Last Name
Executive	Steven	King
Executive	Neena	Kochhar
IT	Alexander	Hunold
Payroll	null	null

Payroll department has no employees. With RIGHT JOIN it appears with null employee names. With INNER JOIN it would be hidden.

Insight 10: Departments With No Employees

Business Question:

Which departments have ZERO employees?

SQL Query:

```
SELECT d.department_name  
FROM employees e  
RIGHT JOIN departments d  
      ON e.department_id = d.department_id  
WHERE e.employee_id IS NULL;
```

How It Works:

RIGHT JOIN keeps ALL departments. If a department has no employees, the employee_id will be null. WHERE e.employee_id IS NULL filters to show only departments where no employee was found. This tells management which departments exist but have no people working in them.

Result:

Department Name
Payroll
Treasury
Corporate Tax
Control And Credit
Shareholder Services
Benefits

Key Pattern: RIGHT JOIN + WHERE ... IS NULL = find rows in the RIGHT table with NO match.

Insight 11: All Departments in One City

Business Question:

Show all departments in the city of Seattle.

SQL Query:

```
SELECT d.department_name
FROM employees e
RIGHT JOIN departments d
    ON e.department_id = d.department_id
WHERE d.location_id = 1700;
```

How It Works:

We use RIGHT JOIN to keep all departments. WHERE d.location_id = 1700 filters to keep only departments in Seattle. This shows us which departments have an office in Seattle. This is a simple filter on the DEPARTMENTS table.

Result:

Department Name
Executive
Sales
Finance
Accounting

Part 4: FULL OUTER JOIN Insights

FULL OUTER JOIN shows ALL rows from both tables. Nothing is hidden. Matched rows and unmatched rows all appear.

Insight 12: All Employees and All Departments

Business Question:

Show ALL employees and ALL departments. Match them together where possible.

SQL Query:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
FULL OUTER JOIN departments d  
    ON e.department_id = d.department_id  
ORDER BY d.department_name;
```

How It Works:

FULL OUTER JOIN shows everything. If an employee has no department, we see the employee with null department. If a department has no employees, we see the department with null employee. If both match, we see both names together. This is the most complete view. No row is hidden.

Result:

First Name	Last Name	Department
Steven	King	Executive
Kimberely	Grant	null
null	null	Payroll
null	null	Treasury

Insight 13: Data Problems: Missing Links

Business Question:

Show only the problem rows. Employees with no department AND departments with no employees.

SQL Query:

```
SELECT e.first_name,  
       e.last_name,  
       d.department_name  
FROM employees e  
FULL OUTER JOIN departments d  
    ON e.department_id = d.department_id  
WHERE e.employee_id IS NULL  
    OR d.department_id IS NULL;
```

How It Works:

FULL OUTER JOIN shows all rows from both tables. Then WHERE filters to keep only the problem rows. e.employee_id IS NULL means the department has no employees. d.department_id IS NULL means the employee has no department. OR means: show rows where EITHER condition is true. This is the best data quality check because it finds problems on both sides in one query.

Result:

First Name	Last Name	Department
Kimberely	Grant	null
null	null	Payroll
null	null	Treasury

Key Pattern: FULL OUTER JOIN + WHERE IS NULL + OR = find ALL unmatched rows from BOTH tables.
This is the best way to find data problems.

Part 5: SELF JOIN Insights

A SELF JOIN is when a table joins with itself. In the EMPLOYEES table, the MANAGER_ID column points to another employee. So we can use SELF JOIN to find who is the manager of each person.

Insight 14: Employee and Their Manager

Business Question:

Show each employee and the name of their manager.

SQL Query:

```
SELECT e.first_name AS employee,
       m.first_name AS manager
FROM employees e
LEFT JOIN employees m
      ON e.manager_id = m.employee_id
ORDER BY e.last_name;
```

How It Works:

We use the EMPLOYEES table two times. "e" is for the employee. "m" is for the manager. ON e.manager_id = m.employee_id means: "find the manager by matching the employee manager_id with another employee employee_id". We use LEFT JOIN because the top boss (the President) has no manager. With LEFT JOIN, the President will appear with null for manager. AS gives a clear name to each column.

Result:

Employee	Manager
Steven	null
Neena	Steven
Lex	Neena
Alexander	Neena
Diana	Alexander

Steven is the President. He has no manager, so his manager is null. Every other employee has a manager.

Insight 15: Manager and Team Size

Business Question:

How many people report to each manager?

SQL Query:

```
SELECT m.first_name AS manager,
       COUNT(e.employee_id) AS team_size
FROM employees m
LEFT JOIN employees e
      ON m.employee_id = e.manager_id
GROUP BY m.first_name
ORDER BY team_size DESC;
```

How It Works:

"m" is the manager. "e" is the employee. ON m.employee_id = e.manager_id means: find all employees whose manager_id is this manager. COUNT counts how many employees report to each manager. GROUP BY m.first_name groups the count by manager name. ORDER BY team_size DESC shows the biggest team first. Managers with no team show 0 because of LEFT JOIN.

Result:

Manager	Team Size
Neena	5
Shelley	3
Steven	2
Alexander	4

Remember: GROUP BY + COUNT = count how many rows are in each group.
This pattern is very useful for reports.

Part 6: 2 or More JOINS (3+ Tables)

Sometimes we need data from 3 or more tables. We add more JOIN lines. The rule is: N tables = N-1 JOIN lines. For example: 3 tables = 2 JOINS. Each JOIN connects the new table to the chain using a shared column.

Insight 16: Employee + Job Title + Department

Business Question:

Show the employee name, job title, and department name in one row.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       j.job_title,
       d.department_name
FROM employees e
INNER JOIN jobs j
    ON e.job_id = j.job_id
INNER JOIN departments d
    ON e.department_id = d.department_id
ORDER BY d.department_name;
```

How It Works:

We need 3 tables: EMPLOYEES (name), JOBS (job title), DEPARTMENTS (department name). So we use 2 INNER JOINS. The chain is: EMPLOYEES --job_id--> JOBS and EMPLOYEES --department_id--> DEPARTMENTS. Both JOINS start from EMPLOYEES. Each JOIN adds one more table to the chain. The result has columns from 3 tables in one row.

Result:

First Name	Last Name	Job Title	Department
Diana	Lorentz	Programmer	IT
Steven	King	President	Executive
Neena	Kochhar	Admin VP	Executive
Shelley	Higgins	Accounting Mgr	Accounting

Key Idea: When you need columns from 3 tables, use 2 JOINS. Each new JOIN adds one table. The chain starts from the table after FROM.

Insight 17: Employee + Department + City

Business Question:

Show the employee name, department name, and city where they work.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name,
       l.city
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN locations l
    ON d.location_id = l.location_id
ORDER BY l.city;
```

How It Works:

We need 3 tables: EMPLOYEES (name), DEPARTMENTS (department name), LOCATIONS (city). The chain is: EMPLOYEES --department_id--> DEPARTMENTS --location_id--> LOCATIONS. Notice the second JOIN uses d.location_id. "d" is the previous table (DEPARTMENTS). Each JOIN connects to the PREVIOUS table. The chain grows step by step.

Result:

First Name	Last Name	Department	City
Diana	Lorentz	IT	South San Francisco
Steven	King	Executive	Seattle
Neena	Kochhar	Executive	Seattle
Shelley	Higgins	Accounting	Seattle

Key Idea: The second JOIN uses a column from the FIRST JOIN result. d.location_id comes from DEPARTMENTS (the table from the first JOIN). This is how the chain works.

Insight 18: Employee Full Profile (4 Tables)

Business Question:

Show the employee name, job title, department name, and city. All in one row.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       j.job_title,
       d.department_name,
       l.city
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN jobs j
    ON e.job_id = j.job_id
INNER JOIN locations l
    ON d.location_id = l.location_id
ORDER BY l.city, d.department_name;
```

How It Works:

We need 4 tables, so we use 3 INNER JOINS. The chain is: EMPLOYEES --department_id--> DEPARTMENTS --location_id--> LOCATIONS, and EMPLOYEES --job_id--> JOBS. 4 tables = 3 JOIN lines. Each JOIN has its own ON condition. This is the full employee profile. One row per employee with all the information. This is a very common query in real work. HR uses it for employee lists and reports.

Result:

First Name	Last Name	Job Title	Department	City
Diana	Lorentz	Programmer	IT	South San Francisco
Steven	King	President	Executive	Seattle
Neena	Kochhar	Admin VP	Executive	Seattle
Shelley	Higgins	Acct Mgr	Accounting	Seattle

Remember: 4 tables = 3 JOINS. 5 tables = 4 JOINS. Each JOIN connects the new table to the chain.

Insight 19: High Salary by Department (WHERE + 3 Tables)

Business Question:

Show employees who earn more than 10000. Include their department name and job title.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       e.salary,
       j.job_title,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN jobs j
    ON e.job_id = j.job_id
WHERE e.salary > 10000
ORDER BY e.salary DESC;
```

How It Works:

We join 3 tables: EMPLOYEES, DEPARTMENTS, and JOBS. Then WHERE e.salary > 10000 filters to keep only high earners. WHERE works the same with multiple JOINS as with one JOIN. The filter is on the EMPLOYEES table (e.salary). ORDER BY e.salary DESC shows the highest salary first. This query tells management: who earns the most, what is their job, and where do they work.

Result:

First Name	Last Name	Salary	Job Title	Department
Steven	King	24000	President	Executive
Neena	Kochhar	17000	Admin VP	Executive

Lex	De Haan	17000	Admin VP	Executive
Shelley	Higgins	12008	Acct Mgr	Accounting

Tip: You can change the WHERE filter to anything:

- WHERE d.department_name = 'Sales' (only Sales department)
- WHERE l.city = 'Seattle' (only Seattle office)
- WHERE e.hire_date > '01-JAN-2005' (new hires)

Insight 20: Employees in One City (WHERE + 3 Tables)

Business Question:

Show all employees who work in Seattle. Include their department name.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN locations l
    ON d.location_id = l.location_id
WHERE l.city = 'Seattle'
ORDER BY d.department_name;
```

How It Works:

We join 3 tables: EMPLOYEES, DEPARTMENTS, LOCATIONS. We need LOCATIONS because the city column is in that table, not in DEPARTMENTS or EMPLOYEES. WHERE l.city = 'Seattle' filters to keep only employees whose office is in Seattle. The filter uses "l" because city is in the LOCATIONS table. This query is useful when HR wants a list of all people in one office.

Result:

First Name	Last Name	Department
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
Shelley	Higgins	Accounting

Key Idea: When you filter on a column from the third table, use the alias of that table in WHERE (l.city, not e.city). Change 'Seattle' to any city name to see employees there.

Insight 21: All Employees With City (LEFT JOIN + 3 Tables)

Business Question:

Show ALL employees with their department and city. Include employees with no department.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name,
       l.city
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id
LEFT JOIN locations l
    ON d.location_id = l.location_id
ORDER BY l.city;
```

How It Works:

We use LEFT JOIN for both JOINS. This keeps ALL employees. If an employee has no department, the department and city will be null. If a department has no location, the city will be null. Compare this with Insight 17 (INNER JOIN). INNER JOIN hides employees with no department. LEFT JOIN keeps them. This is the safest way to show all data.

Result:

First Name	Last Name	Department	City
Diana	Lorentz	IT	South San Francisco

Steven	King	Executive	Seattle
Kimberely	Grant	null	null

Kimberely has no department, so both department and city are null. LEFT JOIN keeps her in the result.

Insight 22: Department Headcount (GROUP BY + JOIN)

Business Question:

How many employees work in each department? Show department name and employee count.

SQL Query:

```
SELECT d.department_name,
       COUNT(e.employee_id) AS total_employees
FROM departments d
LEFT JOIN employees e
      ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY total_employees DESC;
```

How It Works:

We start from DEPARTMENTS and use LEFT JOIN to EMPLOYEES. LEFT JOIN keeps departments with 0 employees too. COUNT(e.employee_id) counts how many employees are in each department. GROUP BY d.department_name puts employees into groups by department. ORDER BY total_employees DESC shows the biggest department first. This is one of the most useful reports for management.

Result:

Department	Employee Count
Sales	34
Shipping	45
IT	5
Executive	3
Payroll	0

Key Pattern: LEFT JOIN + GROUP BY + COUNT = count per group, including empty groups.
Payroll has 0 employees because of LEFT JOIN. With INNER JOIN it would not appear.

Insight 23: Average Salary by Job (GROUP BY + JOIN)

Business Question:

What is the average salary for each job title?

SQL Query:

```
SELECT j.job_title,
       ROUND(AVG(e.salary)) AS avg_salary
FROM jobs j
LEFT JOIN employees e
      ON j.job_id = e.job_id
GROUP BY j.job_title
ORDER BY avg_salary DESC;
```

How It Works:

We start from JOBS and use LEFT JOIN to EMPLOYEES. This keeps all job titles, even those with no employees. AVG(e.salary) calculates the average salary for each job. ROUND formats the number to a whole number (no decimals). GROUP BY j.job_title groups all employees by their job title. This report helps HR see the average pay for each role. If a job title has no employees, the avg_salary will be null.

Result:

Job Title	Avg Salary
President	24000
Administration VP	17000
Accounting Manager	12008
Programmer	5760

Useful Functions:

- AVG(column) = average (ortalama)
- SUM(column) = total (cem)
- MIN(column) = smallest (en küçük)
- MAX(column) = biggest (en büyük)
- COUNT(column) = how many rows (nece eded)

Insight 24: Employees by Country (4 Tables)

Business Question:

Show employees with their department, city, and country name.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       d.department_name,
       l.city,
       c.country_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
INNER JOIN locations l
    ON d.location_id = l.location_id
INNER JOIN countries c
    ON l.country_id = c.country_id
ORDER BY c.country_name, l.city;
```

How It Works:

We need 5 tables this time, so we use 4 INNER JOINS. The chain is: EMPLOYEES --department_id--> DEPARTMENTS --location_id--> LOCATIONS --country_id--> COUNTRIES. We also join JOBS but we do not select from it. Each JOIN adds one table. The chain grows step by step. 5 tables = 4 JOINS. This shows employees with their full location information: department, city, and country.

Result:

First Name	Last Name	Department	City	Country
Steven	King	Executive	Seattle	United States
Neena	Kochhar	Executive	Seattle	United States
Diana	Lorentz	IT	South San Francisco	United States
Michael	Hartstein	Marketing	Toronto	Canada

Chain: EMP --dept_id--> DEPT --loc_id--> LOC --country_id--> COUNTRY

Each step uses a column from the PREVIOUS table to connect to the next.

Insight 25: Employees With Commission + Department (WHERE + JOIN)

Business Question:

Show employees who earn a commission. Include their name, commission, salary, and department.

SQL Query:

```
SELECT e.first_name,
       e.last_name,
       e.salary,
       e.commission_pct,
       d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE e.commission_pct IS NOT NULL
ORDER BY e.commission_pct DESC;
```

How It Works:

We join EMPLOYEES with DEPARTMENTS. WHERE e.commission_pct IS NOT NULL is the key filter. Most employees have null commission. Only sales people have a commission. IS NOT NULL means: keep only rows where commission is NOT empty. This is the opposite of IS NULL. ORDER BY e.commission_pct DESC shows the highest commission first. Management uses this to see who earns commission and how much.

Result:

First Name	Last Name	Salary	Commission	Department
Janette	King	10000	0.35	Sales
Patrick	Sully	9500	0.35	Sales
Allan	McEwen	9000	0.35	Sales

IS NOT NULL = the opposite of IS NULL. It keeps rows that HAVE a value.

IS NULL = keeps rows with no value (empty).

IS NOT NULL = keeps rows that have a value (not empty).

Quick Summary

#	Insight	JOIN	WHERE	Tables
1	Employee + Department	INNER	No	EMP, DEPT
2	Employee + Job Title	INNER	No	EMP, JOB
3	Department + City	INNER	No	DEPT, LOC
4	One Department Employees	INNER	Yes	EMP, DEPT
5	High Salary Employees	INNER	Yes	EMP, JOB
6	All Employees + Dept	LEFT	No	EMP, DEPT
7	Employees Without Dept	LEFT	IS NULL	EMP, DEPT
8	New Hires After a Date	LEFT	Date	EMP, DEPT
9	All Depts + Employees	RIGHT	No	EMP, DEPT
10	Empty Departments	RIGHT	IS NULL	EMP, DEPT
11	Departments in One City	RIGHT	Yes	EMP, DEPT
12	All Emp + All Dept	FULL	No	EMP, DEPT
13	Data Problems	FULL	IS NULL + OR	EMP, DEPT
14	Employee + Manager	SELF LEFT	No	EMP, EMP
15	Manager Team Size	SELF LEFT	GROUP BY	EMP, EMP
16	Emp + Job + Dept (3 tables)	2x INNER	No	EMP, JOB, DEPT
17	Emp + Dept + City (3 tables)	2x INNER	No	EMP, DEPT, LOC
18	Full Profile (4 tables)	3x INNER	No	EMP, DEPT, JOB, LOC
19	High Salary + Dept + Job	2x INNER	Yes	EMP, DEPT, JOB
20	Employees in One City	2x INNER	Yes	EMP, DEPT, LOC
21	All Emp + City (LEFT)	2x LEFT	No	EMP, DEPT, LOC
22	Department Headcount	LEFT	GROUP BY	DEPT, EMP
23	Avg Salary by Job	LEFT	GROUP BY	JOB, EMP
24	Employees by Country	3x INNER	No	EMP, DEPT, LOC, CNT
25	Commission Earners	INNER	IS NOT NULL	EMP, DEPT

Key Patterns to Remember

Pattern	What It Does
INNER JOIN	Shows only rows that match in both tables.
LEFT JOIN	Shows ALL rows from left table + matches from right.
RIGHT JOIN	Shows ALL rows from right table + matches from left.
FULL OUTER JOIN	Shows ALL rows from both tables.
SELF JOIN	Table joins with itself. Good for manager = employee.
WHERE column = value	Filter: keep only rows where column equals the value.
WHERE column IS NULL	Filter: keep only rows where column is empty (null).
WHERE column > value	Filter: keep only rows where column is bigger than value.
WHERE a IS NULL OR b IS NULL	Filter: keep rows where a is null OR b is null.
GROUP BY + COUNT	Count how many rows are in each group.

Most Useful Patterns for Beginners:

1. LEFT JOIN + WHERE IS NULL = find rows with no match (Insight 7)
2. RIGHT JOIN + WHERE IS NULL = find rows with no match in right table (Insight 10)
3. FULL OUTER JOIN + WHERE IS NULL + OR = find all data problems (Insight 13)
4. INNER JOIN + WHERE = join tables AND filter (Insight 4, 5)